



BSc (Hons) in **Information Technology**
Specialized in AI
IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The **state space** is the abstract, fixed structure defined by the problem itself every distinct state (here, every open grid cell) and the transitions between them via legal actions. It exists independently of any search algorithm; it's just "the map." The **search tree** is what the algorithm actually *builds* while exploring its nodes represent *paths from the start state*, not states themselves. Critically, the same state can appear in the search tree multiple times via different paths (a redundant path), which is why the tree can be far larger than the state space, or even infinite in a graph with cycles, even though the underlying state space is finite.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

It solves the **redundant/repeated-state problem**. Without it, the algorithm keeps re-expanding states it has already visited via new paths, which converts a finite state space into a search tree that can blow up exponentially or, worse, loop forever if the state graph has cycles (e.g. two adjacent open cells you can bounce between). This is exactly the tree-search vs. graph-search distinction: adding reached is what makes it graph search. Without it, your DFS agent specifically would risk never terminating in any grid with a cycle (any open room bigger than a single corridor), since DFS has no mechanism to notice "I've already been at this cell via a different route."

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS expands nodes in strict order of path length it fully exhausts all paths of length k before touching any path of length $k+1$ (that's what the FIFO queue guarantees). So the *first* time it reaches the goal, that path is provably the shortest possible in terms of number of steps. Since every move in this grid costs the same (1 step), shortest-in-steps *is* optimal here. DFS has no such ordering it commits to the first neighbor and dives as deep as possible before backtracking, so it can easily stumble onto the food only after wandering far

away first. It's not "wrong," it's just optimizing for depth-first exploration order, not path length, so any path it finds first is essentially incidental.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS's space complexity is $O(b^d)$ it must hold the *entire frontier* at the current depth in memory simultaneously before advancing to the next depth. On a 1000x1000 grid with branching factor ~ 4 , that frontier can grow enormous within just a few dozen steps, well before food is found that's the "crash." UCS has the same exponential space bound when step costs are uniform (it behaves like BFS in that case). DFS's space complexity is only $O(b \cdot m)$ linear in the maximum depth m because it only needs to remember the single current path from the root plus unexplored siblings along that path, not every node at a given depth. So DFS can survive on a huge grid where BFS/UCS exhaust memory the tradeoff being DFS's paths are typically far from optimal, as shown in Q3.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING